



## Unidade 3 — Web Mobile e PWA



## Objetivos de Aprendizagem

Ao final desta aula, o estudante deverá ser capaz de:

- **Configurar** um Manifest completo para uma PWA;
- **Implementar** um Service Worker com estratégias de cache;
- **Aplicar** diferentes estratégias de cache;
- **Analisar** o ciclo de vida de um Service Worker.

**Taxonomia de Bloom:** Configurar, Implementar, Aplicar, Analisar.



## Manifest — Detalhes

```
{
  "name": "TaskFlow - Gerenciador de Tarefas",
  "short_name": "TaskFlow",
  "description": "Gerencie suas tarefas de qualquer lugar",
  "start_url": "/",
  "scope": "/",
  "display": "standalone",
  "orientation": "portrait",
  "background_color": "#ffffff",
  "theme_color": "#C22820",
  "categories": ["productivity", "utilities"],
  "icons": [
    { "src": "/icons/icon-72.png", "sizes": "72x72", "type": "image/png" },
    { "src": "/icons/icon-96.png", "sizes": "96x96", "type": "image/png" },
    { "src": "/icons/icon-128.png", "sizes": "128x128", "type": "image/png" },
    { "src": "/icons/icon-144.png", "sizes": "144x144", "type": "image/png" },
    { "src": "/icons/icon-192.png", "sizes": "192x192", "type": "image/png" },
    { "src": "/icons/icon-512.png", "sizes": "512x512", "type": "image/png" }
  ]
}
```



## Service Worker — Ciclo de Vida

1. REGISTRO  
`navigator.serviceWorker.register('/sw.js')`  
↓
2. INSTALAÇÃO (install)  
Cacheia recursos essenciais  
↓
3. ATIVAÇÃO (activate)  
Limpa caches antigos  
↓
4. FETCH (fetch)  
Intercepta requisições  
Serve do cache ou da rede



## Estratégias de Cache

| Estratégia                    | Descrição                      | Quando usar                           |
|-------------------------------|--------------------------------|---------------------------------------|
| <b>Cache First</b>            | Tenta cache, depois rede       | Recursos estáticos (CSS, JS, imagens) |
| <b>Network First</b>          | Tenta rede, depois cache       | Dados dinâmicos (API, notícias)       |
| <b>Cache Only</b>             | Apenas cache                   | Recursos que nunca mudam              |
| <b>Network Only</b>           | Apenas rede                    | Dados que não podem ser cacheados     |
| <b>Stale While Revalidate</b> | Cache + atualiza em background | Recursos que mudam pouco              |



## Exemplo: Cache First

```
// sw.js – Estratégia Cache First
const CACHE_NAME = 'v1';
const RECURSOS = ['/', '/index.html', '/style.css', '/app.js'];

self.addEventListener('install', (event) => {
  event.waitUntil(
    caches.open(CACHE_NAME)
      .then(cache => cache.addAll(RECURSOS))
  );
});

self.addEventListener('fetch', (event) => {
  event.respondWith(
    caches.match(event.request)
      .then(cached => {
        if (cached) return cached; // Cache hit
        return fetch(event.request); // Cache miss → rede
      })
  );
});

self.addEventListener('activate', (event) => {
  event.waitUntil(
    caches.keys().then(keys => {
      return Promise.all(
        keys.filter(key => key !== CACHE_NAME)
          .map(key => caches.delete(key))
      );
    })
  );
});
```





## Exemplo: Network First

```
// sw.js – Estratégia Network First
self.addEventListener('fetch', (event) => {
  // Apenas para requisições de API
  if (event.request.url.includes('/api/')) {
    event.respondWith(
      fetch(event.request)
        .then(response => {
          // Atualizar cache com resposta da rede
          const clone = response.clone();
          caches.open('api-cache')
            .then(cache => cache.put(event.request, clone));
          return response;
        })
        .catch(() => {
          // Rede falhou → servir do cache
          return caches.match(event.request);
        })
    );
  }
});
```



## Síntese da Aula

### Conceitos principais:

1. O **Manifest** define a identidade e aparência da PWA;
2. O **Service Worker** tem um ciclo de vida: registro → instalação → ativação → fetch;
3. Existem **cinco estratégias** de cache — a escolha depende do tipo de recurso;
4. **Cache First** é ideal para recursos estáticos;
5. **Network First** é ideal para dados dinâmicos.

### Pergunta reflexiva:

Se o Service Worker cacheia recursos, como garantir que o usuário sempre tenha a versão mais recente?

**Próxima aula:** Aula ao vivo — Transformando uma aplicação web em PWA.



